

Okay, I have received your code. Here's a breakdown of the code you provided, explaining each section:

### 1. Imports and Setup

```
from tkinter import messagebox
from tkinter import *
from tkinter import simpledialog
import tkinter
import matplotlib.pyplot as plt
import numpy as np
from tkinter import ttk
from tkinter import filedialog
import pandas as pd
from sklearn.model_selection import train_test_split
import os
import cv2
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score
from sklearn import svm
from sklearn.metrics import accuracy_score
from sklearn.metrics import f1_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
import matplotlib.pyplot as plt
import seaborn as sns
import os
from sklearn.metrics import confusion_matrix

main = Tk()
main.title("Children ADHD Disease Detection using Pose Estimation
Technique")
main.geometry("1300x1200")

global filename
global X, Y
global X_train, X_test, y_train, y_test, scaler, svm_cls
global dataset
proto_File = "Models/pose_deploy_linevec.prototxt"
weights_File = "Models/pose_iter_440000.caffemodel"
n_Points = 18
POSE_PAIRS = [
[1,0], [1,2], [1,5], [2,3], [3,4], [5,6], [6,7], [1,8], [8,9], [9,10], [1,11], [1
1,12], [12,13], [0,14], [0,15], [14,16], [15,17]]
in_Width = 368
in_Height = 368
threshold = 0.1
POSE_NAMES = ["Head", "Neck", "RShoulder", "RElbow", "RWrist",
"LShoulder", "LElbow", "LWrist", "RHip", "RKnee",
"RAnkle", "LHip", "LKnee", "LAnkle", "Chest",
```

"Background"]

```
net = cv2.dnn.readNetFromCaffe(proto_File, weights_File)
net.setPreferableBackend(cv2.dnn.DNN_TARGET_CPU)
```

- **Imports:** This section imports necessary libraries:
  - tkinter: For the graphical user interface (GUI).
  - matplotlib.pyplot, numpy, pandas, seaborn: For data handling, numerical operations, and visualization.
  - sklearn: For machine learning tasks (data splitting, preprocessing, SVM, and performance metrics).
  - cv2 (OpenCV): For image and video processing, particularly for pose estimation.
- **GUI Setup:**
  - main = Tk(): Creates the main window for the application.
  - main.title(...), main.geometry(...): Sets the title and size of the main window.
- **Global Variables:**
  - Declares global variables to store data, model, and file paths. This is a common, though potentially less organized, way to share data between functions in Tkinter applications.
- **OpenPose Model Loading:**
  - proto\_File, weights\_File: Defines the paths to the OpenPose model files (prototxt and caffemodel). These files contain the model architecture and the pre-trained weights, respectively.
  - n\_Points, POSE\_PAIRS, in\_Width, in\_Height, threshold, POSE\_NAMES: These variables define parameters for the OpenPose model, such as the number of keypoints to detect (18), how they are connected to form a skeleton, the input image size for the model, a confidence threshold for keypoint detection, and the names of the keypoints.
  - net = cv2.dnn.readNetFromCaffe(...): Loads the pre-trained OpenPose model using OpenCV's deep neural network (DNN) module.
  - net.setPreferableBackend(cv2.dnn.DNN\_TARGET\_CPU): Configures the model to run on the CPU.

## 2. Functions

- **uploadDataset():**
  - Allows the user to select a CSV file containing the pre-processed pose data using a file dialog.
  - Reads the CSV file using pandas.read\_csv().
  - Displays the file path and the first few rows of the dataset in the text widget.
- **processDataset():**
  - Extracts the features (X) and labels (Y) from the uploaded dataset. It assumes the last column of the CSV is the label.
  - Shuffles the data randomly to prevent bias during training.
  - Normalizes the features using StandardScaler to improve SVM performance.
  - Displays the processed data information in the text widget.
- **splitDataset():**
  - Splits the dataset into training and testing sets (80% for training, 20% for testing) using train\_test\_split().

- Displays the sizes of the training and testing sets.
- **calculateMetrics(algorithm, predict, y\_test):**
  - Calculates and displays various performance metrics for the given algorithm: accuracy, precision, recall, and F1-score.
  - Generates and displays a confusion matrix using seaborn.heatmap and matplotlib.pyplot to visualize the classification results.
- **trainSVM():**
  - Trains an SVM classifier (svm.SVC) with an RBF kernel.
  - Sets hyperparameters like C (regularization) and gamma (kernel coefficient).
  - Fits the SVM model to the training data (svm\_cls.fit(X\_train, y\_train)).
  - Makes predictions on the test data (predict = svm\_cls.predict(X\_test)).
  - Calls calculateMetrics() to evaluate the SVM's performance.
- **predictADHD(testData):**
  - Takes pre-processed pose data as input.
  - Reshapes and scales the input data using the pre-fitted scaler.
  - Uses the trained SVM (svm\_cls) to predict whether the input data represents ADHD or normal behavior.
  - Returns the prediction (0 for normal, 1 for ADHD).
- **detectDisease(frame):**
  - This is the core function for pose estimation and disease detection in a single frame.
  - Takes a video frame as input.
  - Preprocesses the frame for OpenPose.
  - Uses the loaded OpenPose model (net) to detect pose keypoints.
  - Draws circles at the detected keypoint locations and lines connecting them to visualize the skeleton.
  - Extracts the (x,y) coordinates of each detected keypoint and stores them in the testData list. If a keypoint is not detected (confidence below threshold), its coordinates are set to (0,0).
  - Calls predictADHD() to classify the pose data.
  - Returns the processed frame with pose estimation visualization, a skeleton image, and the ADHD prediction.
- **imageDetect():**
  - Allows the user to select an image file.
  - Reads the image using OpenCV.
  - Calls detectDisease() to perform pose estimation and ADHD prediction on the image.
  - Displays the original image with pose estimation overlay and the skeleton image.
  - Shows the prediction result on the image.
- **videoDetect():**
  - Allows the user to select a video file.
  - Reads the video frame by frame.
  - Calls detectDisease() on each frame.
  - Displays the video with pose estimation overlay.
  - Counts the number of frames classified as "normal" and "ADHD".
  - Classifies the entire video based on the majority vote of the frame-level predictions.
  - Displays the overall video classification result.

### 3. GUI Layout

```
font = ('times', 15, 'bold')
title = Label(main, text='Children ADHD Disease Detection using Pose
Extimation Technique')
title.config(bg='darkviolet', fg='gold')
title.config(font=font)
title.config(height=3, width=120)
title.place(x=0,y=5)

font1 = ('times', 13, 'bold')
ff = ('times', 12, 'bold')

uploadButton = Button(main, text="Upload ADHD Pose Dataset",
command=uploadDataset)
uploadButton.place(x=20,y=100)
uploadButton.config(font=ff)
```